

mCOLD device protocol

Version 0.1 (draft) · 2026-10-07 · firmware 0.7.0-dev.33

Status: PROPOSED (protocol 1), 2026-10-02. Written by the firmware side for the iOS app team to accept or change. Nothing here is final until they have; the UUIDs in particular are fixed only once the app ships against them.

The same JSON requests and responses run over every transport. BLE is the first (P5). USB serial carries them now for testing (`rpc` console command), and later USB CDC and anything else can carry them unchanged.

1. Finding the device

1. The app reads the NFC tag: an NDEF record of MIME type `application/json` :

```
{"sn": "MCOLD-9A74", "ble": "28:84:85:27:9A:76", "key": "571ab30e58225355ccfc9b010a935f65"}
```

`sn` and `ble` are the record bring-up wrote; `key` is this box's current authorization key (section 2, *Authorization*). The device keeps the record up to date itself, rewriting only bytes that change. The app team owns this schema; firmware will add a URI record (for the iOS banner) once there is a URL to put in it.

- 1. Tapping the tag also wakes the device's BLE: it advertises for 60 s.
- 2. **iOS never sees a BLE MAC.** The app finds the device by its advertised name, which is the SN, and filters on the service UUID. The MAC in the NDEF record is a tiebreak only, as the app team confirmed.

Advertising: flags + the 128-bit service UUID in the advertisement, the complete local name (`MCOLD-9A74`) in the scan response, 20-40 ms interval while a window is open.

2. BLE GATT

One primary service. All UUIDs share the base `d2a5xxxx-3818-4667-a205-b3ed9c37b8a5` .

UUID	Name	Properties	Security
d2a50000-...	mCOLD service		
d2a50001-...	INFO	read	open
d2a50002-...	STATUS	read, notify	open
d2a50003-...	COMMAND	write	open; what a request may do depends on AUTH
d2a50004-...	RESPONSE	notify	open
d2a50005-...	EVENT	notify	open

- **INFO** is the `GET_INFO` result (section 4), so an app can check the protocol version before sending anything.
- **STATUS** is the `GET_STATUS` result, notified when the trip state or an alarm changes, and after each sample.
- **COMMAND** takes requests, **RESPONSE** returns their answers.
- **EVENT** notifies trip and alarm events as they happen: `{"ev": "ALARM_RAISE", "alarm": "TEMP_HIGH"}` , `ALARM_CLEAR` , `ALARM_ACK` , `TRIP_START` , `TRIP_STOP` .
- **SETTINGS** comes on `EVENT` by itself (decided 2026-10-05): as soon as the app subscribes, again when the session gets AUTH, and whenever a setting changes (from the app, the server or the console) -- so the app's settings page is always the box's: `{"ev": "SETTINGS", "config": {...every key...}, "editable": [...], "trip_locked": [...], "server_rev": 1, "network": {...}}` . `network` (each Wi-Fi network with its DHCP or fixed address, the OTA base; never a password) only after AUTH. About 1.1 KB, so it arrives in several fragments.

Authorization: tap to authorize

Decided 2026-10-02, instead of a passkey on the display. **No BLE pairing**: the phone shows no system prompt, and nobody types anything.

Anyone in BLE range may read INFO and STATUS. Commands that change something (marked 🐞 in section 4) need an **authorized session**, and a session is authorized by proving the app read the NFC tag -- which works only from a few centimetres, so only with the box in hand.

1. Tap: read **key** (16 bytes, hex) from the tag record.
2. Connect, and read **INFO**. It carries **nonce** : 16 random bytes, hex, new for every connection.
3. Send `{"id":..., "cmd": "AUTH", "proof": "<hex>"}` with `proof = HMAC-SHA256(key, nonce)` , both as raw bytes (not as hex text).
4. `{"ok": true}` : this connection may now send any command, until it closes. **NOT_AUTHORIZED** : tap again and retry.

The key never travels over BLE; someone listening sees a nonce and a proof that are useless on any other connection.

The key changes at every reset of the device, and 2 minutes after a session that used it closes. So one tap opens one session (and a link that drops can come back within 2 minutes on the same tap). The device writes the new key to the tag before it starts accepting it, so the tag never offers a key the device would refuse.

Example, from the firmware's own test client:

```
key 7b3066da5096846781a0b5c1d0f6699b
nonce 951c0f73... (from INFO)
proof = HMAC-SHA256(key, nonce) -> {"cmd": "AUTH", "proof": "..."} -> ok
```

Fragments

A message is longer than one BLE write or notification, so both directions use the same framing, independent of the MTU:

```
byte 0 flags: bit 7 FIRST, bit 6 LAST, bits 0-5 fragment index (mod 64)
byte 1.. a piece of the UTF-8 JSON message
```

A message is its FIRST fragment through its LAST, in order (BLE writes with response and notifications are ordered). A single-fragment message has both bits set. A fragment out of sequence, or more than 4,096 bytes in all, discards the message; the device answers **BAD_REQUEST** with `id 0`. A response is cut to fit the negotiated MTU (MTU - 3 bytes of ATT header - 1 byte of flags).

3. Messages

Request:

```
{"id": 17, "cmd": "START_TRIP", "low": 2.0, "high": 8.0}
```

Response:

```
{"id": 17, "ok": true, "trip": 7}
{"id": 17, "ok": false, "err": "ALREADY_ACTIVE", "msg": "a trip is already running"}
```

- `id` : chosen by the app, 1 to 4294967295, unique per request. The response carries it back.
- **Idempotency**. Every command that changes something (marked 🐞) remembers its `id` . Sending the same `id` again returns the original response and does nothing else -- so after a timeout or a dropped connection the app

retries with the same `id`, never with a new one. The last 8 are kept in RAM; `START_TRIP` 's is also kept in flash, so a retry after the device has reset still returns the same trip.

- Unknown keys in a request are ignored, so fields can be added.
- `proto` may be sent in a request; a value the device does not support is answered `UNSUPPORTED_PROTO`.

Errors:

err	Meaning
BAD_REQUEST	not JSON, no <code>id</code> , or no <code>cmd</code>
UNKNOWN_CMD	<code>cmd</code> is not one this firmware knows
BAD_ARGS	an argument missing or out of range
NOT_AUTHORIZED	needs an authorized session (AUTH), or AUTH failed
UNSUPPORTED_PROTO	<code>proto</code> too new
ALREADY_ACTIVE / NOT_ACTIVE	trip state does not allow it
LOG_FULL , NO_LOG , FLASH	storage could not do it
NOT_SUPPORTED	defined, but not in this firmware yet

Units: temperatures in °C as numbers, times as Unix seconds UTC with a separate quality (`none` , `rtc` , `gnss` , `host` , `ntp`), durations in seconds. A value the device does not have is **absent or null , never 0**.

4. Commands

Command	Arguments	Result
GET_INFO		<code>proto</code> , <code>sn</code> , <code>fw</code> , <code>hw</code> , <code>idf</code> , <code>boot</code> , <code>uptime_s</code> ; over BLE also <code>nonce</code> and <code>authorized</code>
AUTH	<code>proof</code> (hex)	authorizes this connection (section 2)
GET_STATUS		see below
GET_CONFIG		<code>config</code> : every setting with its value
SET_CONFIG	<code>key</code> , <code>value</code>	over BLE only the keys of <code>docs/device-settings.md</code> ; the trip-locked ones not while a trip runs
SET_TIME	<code>utc</code> (Unix <code>s</code>)	<code>quality</code>
START_TRIP	<code>low</code> , <code>high</code> (°C); <code>hyst</code> (°C, 0.5), <code>dwell_s</code> (300)	<code>trip</code>
STOP_TRIP		<code>trip</code>
ACK_ALARM		<code>alarms</code> still active
LIST_TRIPS		<code>trips</code> : [<code>"trip"</code> , <code>"last_seq"</code> , <code>"sent"</code>] , oldest first; <code>sent</code> : the server has every row

Command	Arguments	Result
APPLY_CONFIG	 config, wifi, ota_base (each optional; mqtt is refused: console only)	applied, errors: the settings document of docs/device-settings.md, the same one the server sends on mcold/v1/<sn>/config; only the keys listed there
GET_NETWORK		wifi: [{"ssid", "dhcp" [, "ip", "gateway", "subnet", "dns"]}], ota_base: no passwords; needs AUTH
GET_SETTINGS		the SETTINGS event's content, asked for; network only after AUTH
MARK_DELIVERED	 trip	trip, rows: the app gave this finished trip to the server itself; the box will not upload it (ALREADY_ACTIVE while it runs)
GET_TRIP_SUMMARY	trip	thresholds, samples, min, max, alarms, stopped
READ_LOG_CHUNK	trip, from (seq), max (1-16)	records, next
GET_STORAGE_STATUS		sectors, used, free, trips, days_left
SELF_TEST		health: each device and its state
REBOOT		(answered, then the device restarts)
GET_SYNC_STATUS		wifi (connected, ssid, rssi, known: names only), server (broker, pending records, last_ack_s)
SYNC_NOW		upload now rather than at the next pass
SET_WIFI	 ssid, pass (empty for open; may be left out for a known network), dhcp, ip, gateway, subnet, dns	adds a network or changes its password and/or address (DHCP or fixed, as eTEMP has it); up to 5; joins the strongest in range
DEL_WIFI	 ssid	forgets a network
GET_USB_SNAPSHOT_STATUS		NOT_SUPPORTED until the USB drive

GET_STATUS :

```
{
  "id": 3,
  "ok": true,
  "time": {
    "utc": 1790000000,
    "quality": "rtc",
    "boot": 42
  },
  "temp": {
    "ok": true,
    "c": 4.25
  },
  "trip": {
    "active": true,
    "id": 7,
    "samples": 12,
    "min": 3.5,
    "max": 5.25
  },
  "alarms": [
    "TEMP_HIGH"
  ],
  "acked": false,
  "power": {
    "soc": 78,
    "mv": 3987,
    "ma": -12,
    "charge": "none",
    "external": false
  },
  "gnss": {
    "fix": false,
    "age_s": null
  },
  "storage": {
    "used_pct": 1
  },
  "fw": {
    "ver": "0.7.0-dev.1",
    "slot": "ota_1"
  },
  "net": {
    "connected": true,
    "ssid": "Office",
    "rssi": -59,
    "ip": "192.168.1.147",
    "mac": "28:84:85:27:9A:74"
  }
}
```

`fw` is the running firmware and the OTA slot it runs from; `net.ssid`, `rssi` and `ip` are there only while connected. `trip` also carries `alarms_raised` (in the whole trip) and `last_alarm` (`{"type":"HIGH", "utc":...}` or `null`): a server that was out of reach learns from them that an alarm came and went while it could not see.

`READ_LOG_CHUNK` returns records exactly as the device stored them, one JSON object each, `{"seq":5,"type":16,"data":"<base64>"}`, where `data` is the row laid out as in `src/record.h` (record format 3: type 16, every record a ROW -- the same columns the server gets, section 6). `next` is the sequence to ask for next, or `null` when there is no more. The app keeps the byte layout, not the device's interpretation of it -- so a record from a newer firmware is not lost by an older app, only not yet understood.

5. Open questions for the app team

1. Accept or change the UUIDs, the fragment format and JSON.
2. Tap-to-authorize (section 2): the `key` field in the tag record, AUTH with HMAC-SHA256 over the per-connection nonce.
3. Should STATUS be readable without AUTH? It carries temperature and position.
4. The NFC record: keep `{"sn","ble","key"}`, and which URL for the banner?

6. Server: MQTT (for the server team)

Status: PROPOSED, 2026-10-02. The device side is built and running against the team's broker; the server side - above all the ACK -- is what is missing.

The device connects as client id `<sn>` -- the factory serial number, e.g. `mCDV1-L0169-1069-001`; `MCOLD-xxxx` until one is set -- to the broker and login set in its NVS. As eTEMP does it (decided 2026-10-05), **what the box publishes is under `mcold/<sn>/`, what it subscribes to is under `mcold/v1/<sn>/`** -- `v1` is the version of the commands it understands; a box that understands `v2` subscribes there, and both can run on one broker while boxes move over:

Topic	Direction	QoS	Retained	Payload
<code>mcold/<sn>/rec</code>	device → server	1	no	a batch of records
<code>mcold/v1/<sn>/ack</code>	server → device	1	no	<code>{"trip":T,"upto":S}</code>
<code>mcold/<sn>/status</code>	device → server	0	yes	<code>GET_STATUS</code> result (section 4)
<code>mcold/<sn>/online</code>	device → server	1	yes	"1" while connected; "0" when a battery session ends, or from the broker (last will) if the device drops
<code>mcold/v1/<sn>/firmware</code>	server → device	1	yes	a file name to install, e.g. <code>mcold_0.7.1.bin</code> (below)
<code>mcold/v1/<sn>/config</code>	server → device	1	yes	a settings document (<code>docs/device-settings.md</code>), applied once per <code>rev</code>
<code>mcold/<sn>/config/state</code>	device → server	1	yes	what came of it: <code>rev</code> , <code>applied</code> , <code>errors</code> , <code>state</code>
<code>mcold/<sn>/ota/state</code>	device → server	1	yes	what came of it (below)

Subscribing to `mcold/+rec` gets every device's records.

Batches

```
{ "sn": "mCDV1-L0169-1069-001", "trip": 11, "schema": 3, "part": 1, "parts": 1,
  "from": 0, "to": 6, "last": true,
  "rows": [
    { "trip": 11, "seq": 1, "sn": "mCDV1-L0169-1069-001", "timestamp": "02:47:27 05/10/2026",
      "utc": 1791143247, "event": "SAMPLE", "temp": 4.25, "tempmin": 2, "tempmax": 8, "alarm": "",
      "timeok": true, "gnssstate": "last", "latitude": 13.7563, "longitude": 100.5018, "motion": 0,
      "battery": 97, "internet": "online", "detail": "" },
    { "trip": 11, "seq": 2, "sn": "mCDV1-L0169-1069-001", "timestamp": "02:47:28 05/10/2026",
      "utc": 1791143248, "event": "ALARM_HIGH", "temp": 8.5, "tempmin": 2, "tempmax": 8, "alarm": "HIGH",
      ... }, ... ] }
```

Only finished trips are uploaded (decided 2026-10-05, `docs/trip-data-flow.md`): while a trip runs its rows stay in the box and the server gets the `status`; once it has ended it goes up in parts of 20 rows, oldest trip first. Part `k` always holds `seq 20(k-1)..20k-1`, `parts` is how many the trip has, and `last` marks the one with `TRIP_STOP`. A trip the app has already delivered (`MARK_DELIVERED`) is not uploaded at all.

Every record is a row with the same columns (record format 3, decided 2026-10-05): a sample and every event alike carry the state of the box at that moment, so the server stores one table, and the CSV a person opens has the same columns (`trip csv` at the console prints one, `src/logrow.h` defines it).

Column	Meaning
<code>trip</code>	the trip number, made by the box (unique with <code>sn</code>)
<code>seq</code>	the row's place in the trip, from 0; (<code>sn</code>, <code>trip</code>, <code>seq</code>) is the key
<code>sn</code>	the box's serial number
<code>timestamp</code>	local time, hh:mm:ss DD/MM/YYYY (config <code>tz_offset_min</code> , +07:00 by default)
<code>utc</code>	the same moment in Unix seconds
<code>event</code>	what the row is (below)
<code>temp</code>	°C, calibrated; <code>null</code> when the probe gives none
<code>tempmin</code> / <code>tempmax</code>	the trip's alarm limits, from its start
<code>alarm</code>	alarms active after this row: <code>HIGH</code> , <code>LOW</code> , <code>PROBE</code> , <code>BATTERY</code> , joined with <code> </code> ; empty for none
<code>timeok</code>	false when the box did not know the time; then <code>timestamp</code> / <code>utc</code> are <code>null</code> and <code>boot + up_s</code> order the row
<code>gnssstate</code>	<code>fix</code> (within the last sample period), <code>last</code> (an older fix), <code>none</code>
<code>latitude</code> / <code>longitude</code>	degrees; <code>null</code> with <code>none</code>
<code>motion</code>	motion events since the previous row
<code>battery</code>	state of charge, %; <code>null</code> unknown
<code>internet</code>	how the last upload attempt went: <code>online</code> , <code>wifi_only</code> (Wi-Fi, no broker), <code>offline</code>
<code>detail</code>	per event, in words (below); usually empty

event	When	detail
TRIP_START / TRIP_STOP	the trip begins / ends	
SAMPLE	every sample period	
ALARM_HIGH / ALARM_LOW	outside tempmax / tempmin for longer than the dwell	
ALARM_PROBE	no temperature for over a minute	
BATTERY_LOW	battery under 15 %	the %
ALARM_CLEAR	an alarm is over	which
ALARM_ACK	someone acknowledged	the alarms
PROBE_FAULT / PROBE_OK	the probe stops / starts answering	the fault
USB_IN / USB_OUT	external power plugged in / pulled out	
POWER_ON	the trip carried on after a reset	reset reason
POWER_OFF	battery empty: the box switched itself off	cell mV
TIME_SET	the clock was set	the time before
DATA_LOST	a trip deleted because the log was full	its trip number
SHOCK	reserved: a shock above a threshold (none set yet)	

Ordinary motion is not a row: it is counted in `motion`. Trips recorded before format 3 are not uploaded.

The ACK -- what the server must do

The device deletes nothing until the server says it has stored it (§9.5 of the requirements). **An MQTT PUBACK does not count:** it only means the broker took the message.

1. Store the batch's records durably. **The key is (sn, trip, seq):** a record that arrives twice -- the device resends whenever an ACK does not come -- must be stored once, not twice.
2. Then publish to `mco1d/v1/<sn>/ack`:

```
{"trip":8,"upto":15}
```

meaning **every record of trip 8 with seq 0..15 is stored**. It is a high-water mark, so it must be contiguous: only acknowledge `upto` when all records below it are stored too.

The device checks every ACK against its own log and rejects one for a trip it does not have, or past the last record it holds; a rejected ACK marks nothing. The mark only ever moves forward, so a late or repeated ACK is harmless.

Timing on USB power (always connected): after a batch the device waits 15 s for the ACK, then resends, doubling the wait up to 5 minutes while the server stays silent, and going back to 15 s at the next ACK. The status goes on every change, every five minutes, and at once whenever the broker answers again.

On battery: sessions, not a connection

On battery the box sleeps between samples and Wi-Fi is off. While a trip runs, or rows of a finished one are waiting, it connects for a **session** once per `upload_period_s` (default 300 s, one sample period): Wi-Fi up, broker connect, `status`, batches, then `"0"` on `online` and a clean disconnect. Inside a session it waits **10 s** for each ACK; an ACK later than that is too late for the session, and the batch goes again next time. A session that gets no ACK at all doubles the wait before the next one (10, 20, 40 ... minutes, up to 4 hours) -- a server that is not answering would otherwise cost a session of radio every five minutes.

So, for the server:

- **ACK fast.** Each second the server takes is a second of radio on every box, every session; an ACK within one or two seconds of the batch keeps a session around 5 s. No ACK means the box backs off and data arrives hours late.
- Treat a box as reachable by when its last `status` arrived, not by `online`: a sleeping box is `"0"` most of the time and is fine.
- An ACK published after the session has ended is lost (the box connects with a clean session); the box sends that batch again next time, and the server, keyed on (sn, trip, seq), simply ACKs it again.

`tick` in the record stamp is milliseconds on a clock that runs through sleep, restarted only by a real reset (which also increments `boot`); together they order every record of a device.

When the log fills, the device first deletes trips the server has acknowledged in full -- no data lost. Only if there are none does it delete the oldest unacknowledged trip, and then it records the loss.

Firmware updates (OTA)

Built and tested 2026-10-04, the way eTEMP V2 does it: the server names a file, and the box downloads it itself.

1. Put the image on the file server, in the folder the box knows (its base URL, in NVS; `ota base URL` at the console). The build leaves it as `.pio/build/mcold/mCOLD_<version>.bin`.
2. Publish the file name to `mcold/v1/<sn>/firmware`, **retained** -- a box on battery is asleep almost all the time and only sees it at its next session. A whole `https://...` URL is accepted too.
3. Watch `mcold/<sn>/ota/state`:

```
{ "ver": "0.7.1", "state": "downloading", "pct": 40 }
{ "ver": "0.7.1", "state": "rebooting", "from": "0.7.0" }
{ "ver": "0.7.1", "state": "ok", "from": "0.7.0" }
{ "file": "mCOLD_0.7.1.bin", "state": "failed", "reason": "..." }
{ "file": "mCOLD_0.7.1.bin", "state": "deferred", "reason": "trip" }
{ "ver": "0.7.1", "state": "rolled_back", "running": "0.7.0" }
{ "file": "...", "ver": "0.7.0", "state": "skipped", "reason": "not newer", "running": "0.7.0" }
```

1. After `ok`, clear the retained message (publish an empty retained payload). A name the box has already handled is ignored anyway, so leaving it costs nothing but a few bytes per session.

The box checks everything itself; the server needs to know none of it: the image header (same project, newer version), the image's SHA-256, the **signature** (RSA-3072, checked against the key of the image it is running -- an image not signed with the project key is refused), and after the reboot it must reach the broker within 3 minutes or the bootloader goes back to the old image, which then says `rolled_back`. It waits (`deferred`) while a trip runs or the battery is under 30 %, and goes ahead by itself once both allow. On battery with nothing to send it still checks in every 6 hours, so a box between trips sees a firmware message too.

Open questions for the server team

1. Accept the topics and the ACK (or say what to change).
2. **TLS on 8883 and per-device logins.** On 1883 everything, login included, crosses the network in clear, and one shared login means anyone holding it can publish as any box -- including false ACKs that make devices delete data they never delivered. §10.4 asks for TLS.
3. ~~OTA: a file server or MQTT?~~ Answered by the eTEMP way: the server sends the file name, the box fetches it (above).